K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Batch: B2	**Roll No.: 16010124107**
Experiment No.7	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Title:Study, Implementation, and Analysis of 0/1 Knapsack Problem.

Objective: To learn 0/1 Knapsack Problem using Dynamic Programming Approach. (Maximize the total value of selected items while ensuring their total weight does not exceed W.)

CO to be achieved:

CO 2 Describe various algorithm design strategies to solve different problems and analyse Complexity.

Books/ Journals/ Websites referred:

1. **Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press**
2. **T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algorithms",2nd Edition ,MIT press/McGraw Hill,2001**
3. **http://www.lsi.upc.edu/~mjserna/docencia/algofib/P07/dynprog.pdf**
4. **http://www.geeksforgeeks.org/travelling-salesman-problem-set-1/**
5. **http://www.mafy.lut.fi/study/DiscreteOpt/tspdp.pdf**
6. **https://class.coursera.org/algo2-2012-001/lecture/181**
7. **http://www.quora.com/Algorithms/How-do-I-solve-the-travelling-salesman-problem-using-Dynamic-programming**
8. **www.cse.hcmut.edu.vn/~dtanh/download/Appendix_B_2.ppt**
9. **www.ms.unimelb.edu.au/~s620261/powerpoint/chapter9_4.ppt**

Pre Lab/ Prior Concepts:

Data structures, Concepts of algorithm analysis

Historical Profile:

Dynamic Programming (DP) is used heavily in optimization problems (finding the maximum and the minimum of something). Applications range from financial models and operation research to biology and basic algorithm research. So the good news is that understanding DP is profitable. However, the bad news is that DP is not an algorithm or a data structure that you can memorize. It is a powerful algorithmic design technique.

The 0/1 Knapsack Problem is one of the most studied problems in computer science, operations research, and combinatorial optimization. Its origins and development reflect the evolution of mathematical optimization and computational techniques.

Historical Background

Origins in Resource Allocation:The knapsack problem has its roots in resource allocation problems dating back centuries, where individuals needed to maximize their gain (value) from limited resources (capacity).The term "knapsack" derives from the idea of filling a knapsack with items to achieve the greatest benefit without exceeding its weight limit.

Early Formalization: The problem was first mathematically formalized in the early 20th century as part of broader studies in combinatorics and optimization. It gained attention as a theoretical challenge in discrete mathematics.

Greedy Algorithms and Limitations:Researchers also explored greedy approaches for simplified variants (e.g., fractional knapsack).The greedy algorithm does not work for the 0/1 knapsack problem due to its inability to handle binary choices optimally.

New Concepts to be learned:

Application of algorithmic design strategy to any problem, dynamic Programming method of problem solving Vs other methods of problem solving, optimality of the solution, Optimal Binary Search Tree Problems and their applications

---

Theory:

0/1 Knapsack is a dynamic programming problem that selects items to maximize value without exceeding capacity. Each item is either taken or not taken. It avoids greedy limitations by checking all combinations efficiently using optimal substructure and overlapping subproblems to produce an optimal solution.

Algorithm:

1. Initialize DP table.
2. Rows for items, columns for weight.
3. If item fits, take max of include or exclude.
4. Else copy above value.
5. Repeat for all items.
6. Final answer at last cell.

Example:

Consider the following example with four items

item	item1	item2	item3	item4

profit	10	15	40	25
weight	2	3	5	4

Let maximum weight = 6

Solution for the example:

Weight	0	1	2	3	4	5	6
0 items	0	0	0	0	0	0	0
item1	0	0	10	10	10	10	10
item2	0	0	10	15	15	25	25
item3	0	0	10	15	15	40	40
item4	0	0	10	15	25	40	40

Analysis of algorithm:

The 0/1 Knapsack uses dynamic programming to build a table of size n×W. Each entry dp[i][w] represents the maximum profit using first i items and capacity w.

The recurrence (relaxation) is:
If weight fits, take a maximum of excluding or including the item.
Otherwise, copy the previous value.

Relaxation formula (DP relation):
$dp[i][w] = \max(dp[i-1][w], profit[i] + dp[i-1][w - weight[i]])$

Condition: $weight[i] \leq w$

If not:
$dp[i][w] = dp[i-1][w]$

Thus each state is computed once, giving time complexity O(n*W) and space O(n*W). This is efficient compared to exponential brute force.

CONCLUSION:

The 0/1 Knapsack problem is efficiently solved using dynamic programming, which ensures an optimal solution under given weight constraints. The DP approach avoids repeated calculations by storing intermediate results in a table. It guarantees maximum profit selection with time complexity O(nW), making it more efficient than brute force methods.

Topic: Backtracking

Theory: In many applications of the backtrack method, the desired solution is expressible as an n-tuple *(x1,...,Xn),* where the x*i* are chosen from some finite set Si. Often the problem to be solved calls for finding one vector that maximizes (or minimizes or satisfies) a *criterion function P(x1,..... . , xn). Sometime*s it seeks all vectors that satisfy *P.* For example, sorting the array of integers in. *a[1* : n] is a problem whose solution is expressible by an *n- tuple, w*here x*i* is the index in *a* of the ith smallest element. The criterion function P is the inequality *a[xi]* ≤ *a[xi+1]* for 1 ≤ i < *n.* The set *Si* is finite and includes the integers 1 through *n.* Though sorting is not usually one of the problems solved by backtracking, it is one example of a familiar problem whose solution can be formulated as an n-tuple.

Control abstraction:

```
void Backtrack( int k )

// This is a schema that describes the backtracking process //using recursion. On entering, the
first k-1 values x[1], x[2], //…., x[k-1] of the solution vector  x[1:n] have been //assigned. x[]
and n are global.
{
        for (each x[k] such that x[k] Є T(x[1], …, x[k-1])
    {

            if (Bk  (x[1], x[2], …, x[k]))
             {

                if (x[1], x[2], …, x[k] is a path to an answer node)

                        output x[1:k];

                if (k < n) Backtrack(k+1);
            }
        }

    }
```